adani | Institute of Digital Technology Management

A Case Study on the Use of Image Processing and Computer Vision at Tesla

Subject: Image Processing using Python

Case Study Report

Student Name: Chirag Ubhadiya

Enrollment Number: 20251014

Institute: Adani Institute of Digital Technology Management

1. Abstract

Driver-assistance systems can only be as good as their understanding of the road, and that understanding begins with images. This case study looks at how image processing and computer vision are used in Tesla vehicles, which since 2021 have moved step by step to a camera-based approach that the company calls Tesla Vision [1]. The report keeps three kinds of information apart: what Tesla documents publicly, what Tesla engineers have said in public conference talks, and what is ordinary computer-vision practice that helps explain how a system of this kind can work. Tesla's production software is proprietary, so the report does not claim to describe its internal design.

The techniques covered are grayscale and colour-space processing, filtering, Sobel, Laplacian and Canny edge detection, object detection, semantic segmentation, motion estimation, tracking and deep multi-camera networks. A small Python and OpenCV experiment on a synthetic road image (Figure A3) shows how the three edge detectors behave on noisy data. Three findings stand out. First, classical image processing is still useful for understanding and for pre-processing, but learned models now carry most of the perception work. Second, Tesla's manuals and regulators' documents agree that camera performance depends on lens cleanliness, calibration, lighting and weather, and that the features are driver-assistance functions that need constant supervision [2, 3]. Third, rare and degraded-visibility situations remain the hardest part of the problem, and they are the subject of an ongoing U.S. safety investigation [4]. The report closes with a balanced view of benefits, limits and future directions.

Keywords: image processing, computer vision, Tesla Vision, driver assistance, object detection, semantic segmentation, OpenCV.

2. Introduction

2.1 Image processing and computer vision

Image processing and computer vision are closely related but they answer different questions. Image processing takes an image and produces another image, or a set of simple measurements, usually to make the picture easier to analyse. Examples are converting a colour photo to grayscale, smoothing noise, sharpening, adjusting contrast and finding edges [5]. Computer vision goes a step further and tries to extract meaning: which pixels belong to a road, whether a shape is a pedestrian, how far away an object is and where it is moving [6]. In practice the two are used together. A typical pipeline cleans up the image first and then hands it to a detector or a neural network.

For a student working in Python the distinction is easy to see in code. Calls such as *cv2.cvtColor*, *cv2.GaussianBlur* and *cv2.Canny* from the OpenCV library [7] are image-processing operations. A trained network that outputs "pedestrian, 0.93" for a region of the picture is computer vision. This report uses both terms, and it tries to say clearly which one is meant.

2.2 Why vision matters in modern vehicles

Human drivers rely mostly on sight. They read lane markings, notice brake lights, judge the speed of other cars and react to people at the roadside. A driver-assistance system that wants to help with any of these tasks needs a machine equivalent of that sight. Cameras are attractive for this because they capture colour, texture and shape at high resolution and because a traffic sign, a lane line or a traffic light is designed to be read visually. The cost of getting it wrong is high, so image processing in a car is not only an accuracy problem but also a safety problem.

2.3 Purpose and scope

The purpose of this case study is to explain, at postgraduate level, how image processing and computer vision relate to Tesla's camera-based driver-assistance features and to evaluate the approach critically. The scope is limited to publicly available material: Tesla's owner's manuals and support pages, documents published by the U.S. National Highway Traffic Safety Administration (NHTSA), conference talks and papers, and standard textbooks. I did not have access to Tesla's software, data or engineers, and I did not run any experiment on Tesla vehicles. Where I describe a general technique I say that it is general, and where Tesla's practice is not public I say that too. The only experiment in the report is the small OpenCV comparison in Figure A3, which uses a synthetic image and says nothing about Tesla's performance.

Section 3 introduces the company, Section 4 states the perception problem, and Sections 5 and 6 explain the workflow and the techniques. Section 7 covers documented applications, Section 8 covers data and evaluation, Sections 9 and 10 weigh benefits against limitations, and Sections 11 and 12 look ahead and conclude. Figures and tables are collected in Appendices A and B.

2.4 Method and source quality

I sorted sources into tiers. The most reliable are Tesla's owner's manuals and support pages for statements about the product, and documents published by NHTSA for statements about regulatory action. Next come textbooks and peer-reviewed papers for the general techniques. Public conference talks are used with care, since they describe ideas at a point in time and not the current production software. I did not use blog posts, forum threads or social-media statements as evidence for technical claims, because they are often impossible to verify and they change quickly. Tesla's manuals

are written per model, region and software version, so where wording differs I relied on the Model Y or Model S pages and flag that features may vary. All web sources were checked in October 2026, and Tesla software and regulatory proceedings can change after that date.

3. Company Background

Tesla, Inc. is an American manufacturer of electric vehicles. Its current range includes the Model S, Model 3, Model X, Model Y and Cybertruck [3]. Alongside the vehicles, Tesla sells a family of driver-assistance features. According to the owner's manual, Traffic-Aware Cruise Control is fitted to all Tesla vehicles and keeps a set speed and an adjustable following distance from the car ahead, while the broader suite called Full Self-Driving (Supervised) is an option whose availability depends on the market, vehicle configuration and software version [8, 9]. The older name Autopilot still appears in regulatory documents, so the report uses it where the source does.

The company's sensing strategy changed over time. Tesla states that it began removing radar from the Model 3 and Model Y in 2021 and from the Model S and Model X in 2022, and that it later removed ultrasonic sensors, starting with the Model 3 and Model Y in 2022 and followed by the Model S and Model X in 2023 [1]. Together with the removal of ultrasonic sensors, Tesla introduced a vision-based "occupancy network" to supply the information those sensors had provided [1]. The result is that, in most regions, Tesla's driver-assistance features now depend on cameras and neural-network processing rather than on radar or ultrasonic range sensing.

This makes Tesla a useful case for a course on image processing for three reasons. The sensing choice is unusually explicit: Tesla describes its system as camera-based [1], so the quality of image interpretation is central, not one input among many. Tesla's engineers have also given public talks on parts of the perception design [10, 11]. Finally, because the features are used on public roads at large scale, regulators have published detailed documents about them [4, 12]. That combination gives a student rare access to both the technical ambition and the documented problems. Choosing Tesla as the example is not an endorsement. Other manufacturers combine cameras with radar or lidar, and that alternative is discussed in Section 10.

4. Problem Statement and Need for Image Processing

The central problem is to turn a stream of two-dimensional images into a dependable description of a three-dimensional, moving, partly unpredictable world. A camera projects that world onto a flat sensor and in doing so discards direct depth information. A pedestrian two metres away and a small poster of a pedestrian twenty metres away can look similar in a single frame, and the system has to resolve that ambiguity from visual cues, from several cameras and from motion over time. It must also do all of this fast enough to be useful, because a stale description of the road is a dangerous one.

Road users. Pedestrians are articulated, wear many different clothes, and appear at very different sizes in the image depending on distance. Cyclists and motorcyclists are narrow and easily hidden. Vehicles range from small hatchbacks to long trucks, and a vehicle that is partly out of frame still needs to be recognised. **Lane markings and road edges.** Markings can be worn, faded, covered by water or snow, repainted, or contradicted by temporary construction markings. Tesla's own manual notes that faded lane markings and rain can affect performance, and that the cameras need highly visible markings when they calibrate [2, 13]. **Traffic signs and signals.** Signs differ between countries, can be partly hidden by branches, and traffic lights can be confused with other bright objects at night. Tesla's manual states that Full Self-Driving (Supervised) depends on the cameras being able to detect traffic lights, stop signs and road markings [9].

Lighting. A camera has a limited dynamic range. Low sun, tunnel exits, oncoming headlights and reflections from wet roads can saturate part of the image while another part stays almost black. Night driving adds sensor noise, because the sensor has to amplify a weak signal. **Weather.** Rain drops on the lens, fog and snow reduce contrast and hide detail. The Tesla manual lists heavy rain, snow, fog and hail among the conditions that can limit the features, and it warns that an obstructed or blinded camera can make them unavailable [2]. **Occlusion.** A child stepping out from behind a parked van is visible for only a fraction of a second, and the system has to react to a partial view. **Unusual scenes.** Overturned vehicles, animals, fallen loads and unusual road works are rare, which means a learning-based system has seen few examples of them.

Geometry and time. Two further constraints make the task harder than it first appears. The first is perspective: objects shrink with distance, so a pedestrian far ahead may be only a few dozen pixels tall, and a small error in pixel position then becomes a large error in estimated distance. A system with a single camera view can use cues such as the known size of vehicles, the point where an object touches the road surface, and motion over several frames, and overlapping cameras add more information. All of these depend on accurate calibration. The second constraint is time. A car travelling at 100 km/h covers about 28 metres every second, so a perception delay of 100 milliseconds corresponds to nearly three metres of travel before the system has even finished looking. Accuracy and speed therefore have to be achieved together, which limits how large a model can be on the vehicle's own computer.

These difficulties explain why image processing is needed and also why it is hard. Simple rules, such as "a white line is a pixel band with high brightness", work on clean test images but fail when brightness changes, when paint is worn, or when another bright object resembles a lane line. The field has therefore moved from hand-designed filters towards

learned models that can cope with variation, while still using classical processing for tasks such as noise reduction and normalisation [6, 14]. Section 5 describes the general workflow, and Section 6 describes the techniques in more detail.

5. Image Processing and Computer Vision Workflow

This section describes a general perception workflow for a camera-based driving system (Figure A1). It is a teaching model assembled from standard computer-vision practice. It is not Tesla's proprietary architecture, which is not public, and modern networks often merge several of these stages into a single model. Where Tesla has said something publicly I mark it and give the source. Figure A2 shows the same stages applied to a synthetic scene, with schematic annotations rather than the output of any real model.

5.1 Camera image acquisition

The first stage is capturing frames. A camera sensor converts light into electrical signals, and the camera electronics turn these into a digital colour image through steps such as demosaicing, exposure control and white balance. Exposure matters a great deal on the road: too long and moving objects blur, too short and dark scenes become noisy. Tesla's manuals state that the vehicle's cameras are mounted at the front, rear, left and right and are used to build a model of the area around the car [9]. In a public talk, a Tesla engineering leader described a network that takes in eight camera streams [10]. Cameras must also be calibrated so that pixel positions can be related to directions in the real world, and Tesla's manual asks the driver to drive a short distance on well-marked roads so that the cameras can calibrate themselves [13].

The link between pixels and the real world is usually described by the pinhole camera model. A point at position (X, Y, Z) in camera coordinates projects to the image position

$$u = f_x \cdot (X / Z) + c_x, \qquad v = f_y \cdot (Y / Z) + c_y,$$

where f_x and f_y are the focal lengths in pixels and (c_x, c_y) is the image centre [6]. The division by Z is the reason depth cannot be recovered from one point in one image: every point along the same ray gives the same pixel. Calibration estimates these parameters and the camera's orientation on the vehicle, so that image positions can be turned into directions on the road. In OpenCV this is the territory of functions such as *cv2.calibrateCamera* and *cv2.undistort*. A camera that has moved or been knocked slightly changes these numbers, which is one reason why Tesla asks for recalibration after servicing and in some cases after software updates [13].

5.2 Pre-processing and normalisation

Raw frames are rarely fed to later stages unchanged. Typical steps are resizing, cropping, correcting lens distortion, reducing noise and normalising brightness and contrast so that one network sees consistent input across day and night. Classical operations such as Gaussian filtering and contrast-limited adaptive histogram equalisation (CLAHE) [15] are examples (Figure A2, panel 2). In deep-learning systems some of this is learned inside the first layers of the network, so the line between pre-processing and recognition is blurred. Whether Tesla applies particular classical filters before its networks is not publicly documented.

5.3 Object detection and classification

Detection answers two questions: *what* is in the frame and *where*. Detectors output a bounding box and a class label, such as vehicle, pedestrian, cyclist or traffic sign, together with a confidence score. Classification of the detected region can then add detail, for example whether a traffic light is red or green, or what number is on a speed-limit sign. Detection is the stage that gives the downstream system its list of objects to worry about, which is why missed detections and false detections are both important (Section 10).

5.4 Lane and road-region identification

The system also needs to know where it can drive. Lane identification finds the lane lines and road edges and estimates the shape of the lane ahead. Road-region identification is broader and labels the drivable surface, including places where there are no markings at all. Classical methods used edge detection followed by a Hough transform to find straight lines, and learned methods now treat lane finding as a segmentation or instance-segmentation problem [16]. The Tesla manual states that the system follows curves in the road and uses map data to improve accuracy, although it is primarily based on camera data [9].

5.5 Image segmentation

Segmentation assigns a class to every pixel, such as road, sidewalk, vehicle, person, sky or vegetation. It gives a denser description than bounding boxes, because it can show the exact shape of the drivable area and objects with irregular outlines (Figure A2, panel 4). Fully convolutional networks and encoder–decoder designs made this practical [17, 18], and public benchmarks such as Cityscapes provide labelled street scenes for training and testing [19].

5.6 Object tracking across video frames

A single frame shows where things are, but driving depends on where they are going. Tracking links detections from one frame to the next, so that "the car in the right lane" keeps the same identity and its motion can be estimated. This gives speed and direction, smooths out flickering detections, and helps the system keep an object in mind for a moment

when it is briefly hidden. A common formulation uses a motion model such as a Kalman filter and then matches predictions to new detections [20, 21] (Figure A2, panel 5).

5.7 Scene interpretation

Scene interpretation combines the outputs of the earlier stages into one consistent picture: which lane the car is in, which objects are close, what the traffic signals say, and which objects might move into the car's path. With several cameras the system also has to merge overlapping views into a common frame of reference (Figure A2, panel 6). Tesla has described one such representation publicly. At a 2022 conference workshop its engineers presented an *occupancy network* that takes the multi-camera images and predicts which parts of the three-dimensional space around the car are occupied, together with how they move [10, 11]. Tesla's support page also says that the occupancy network was launched alongside the removal of ultrasonic sensors to replace the inputs those sensors generated [1]. These sources describe the idea, not the exact production implementation, which may differ and which changes with software versions.

5.8 Delivery to downstream driving functions

The final stage passes the interpreted scene to the parts of the system that plan and act: speed control, steering support, braking, and the display and alerts that the driver sees. This is where the camera output becomes a physical action, so errors have the most serious consequences here. The manual warns that the on-screen visualisation may not show everything the cameras are using [9], which is a reminder that the display is a summary rather than a complete view of the system's perception.

5.9 What is verified and what is illustrative

To summarise the status of the material above: the use of cameras around the vehicle, the camera-based replacement of radar and ultrasonic sensors, the calibration requirement and the sensitivity to weather and obstruction are documented by Tesla [1, 2, 13]. The occupancy-network idea comes from a public talk summarised in the literature [10, 11]. The eight-stage breakdown, the use of specific classical filters and the tracking method are general computer-vision explanations used to make the system understandable.

6. Image Processing Techniques

This section explains the techniques that a student should know when thinking about a driving scene. For each one I describe its purpose, how it works at a high level, and how it could help interpret a road scene. I do not claim that Tesla uses each of them directly in production. Table B1 collects them in one place.

6.1 Grayscale conversion and colour-space processing

A colour image stores three values per pixel, usually red, green and blue. Converting to grayscale reduces this to one brightness value, commonly a weighted sum such as $0.299R + 0.587G + 0.114B$, which reflects the eye's greater sensitivity to green [5]. The benefit is less data and simpler processing for steps like edge detection. Other colour spaces separate brightness from colour: in HSV or YCrCb a yellow lane line can be described mainly by hue and saturation, which makes it less sensitive to a change in overall brightness than a raw RGB threshold. In a driving scene this helps to find white and yellow markings, red tail lights and the colour of a traffic light. The cost is that throwing away colour also throws away information, and a grayscale image cannot tell a red signal from a green one of similar brightness.

6.2 Image filtering, thresholding and morphology

Filtering replaces each pixel by a combination of its neighbours. A Gaussian filter averages with weights that fall off with distance, which suppresses random sensor noise at the price of some blur. A median filter replaces a pixel with the median of its neighbourhood and works well against isolated bright or dark specks. A bilateral filter smooths flat regions but keeps strong edges, because it also weighs neighbours by similarity in intensity [22]. Histogram equalisation, and its adaptive form CLAHE, stretch the contrast of dim regions [15]. These steps matter at night and in rain, when images are noisy or flat. They also introduce trade-offs: too much smoothing removes thin features such as distant lane lines, and strong contrast enhancement can amplify noise.

Two further classical tools are often used after filtering. *Thresholding* turns a grayscale or single-channel image into a binary mask by keeping pixels above or below a chosen value. A single global threshold fails when lighting varies across the image, so adaptive thresholding computes a local threshold for each neighbourhood, and Otsu's method picks a global threshold automatically from the histogram [5]. *Morphological operations* then clean the mask using a small structuring element: erosion removes thin specks, dilation fills small gaps, opening (erosion then dilation) removes noise, and closing (dilation then erosion) joins broken parts such as the gaps in a dashed lane line. In a driving scene these steps can turn a rough bright-pixel mask for lane paint into compact, connected shapes that are easier to fit with lines or curves. They also show the weakness of classical methods, since every threshold and kernel size is a tuned constant that may suit one road and light level but not another.

6.3 Edge detection using Sobel, Laplacian and Canny

Edges are places where brightness changes sharply, and they often coincide with the boundaries of objects, lane lines and signs. The **Sobel** operator estimates the horizontal and vertical brightness gradients using two 3×3 kernels, which combine smoothing with differentiation [23]. The gradient magnitude and direction are then

$$G = \sqrt{(G_x^2 + G_y^2)}, \quad \theta = \arctan(G_y / G_x).$$

The **Laplacian** is a second-derivative operator, $\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$. An edge appears as a place where the second derivative crosses zero, but because the operator is a second derivative it reacts strongly to noise and gives no direction [5, 24]. The **Canny** detector is a multi-step method: Gaussian smoothing, gradient computation, non-maximum suppression to thin the edges, and hysteresis thresholding with a high and a low threshold, so that weak edges are kept only when they connect to strong ones [25, 26].

Figure A3 compares the three on a synthetic road image with added noise, produced with OpenCV in Python. The comparison shows what theory predicts. The Laplacian applied without smoothing (panel d) is dominated by noise, while the same operator after a Gaussian blur (panel e) gives usable but double-lined edges. Sobel (panel b) gives bright, thick responses at the lane lines, the vehicle outline and the sign. Canny (panel c) gives thin, single-pixel edges and ignores most of the noise, and lowering its thresholds (panel f) keeps more weak outlines such as the distant buildings. In a driving system, edges can provide lane-line candidates and object boundaries, but a fixed threshold that works in sunlight may fail in fog, which is one reason modern systems rely more on learning than on hand-tuned edge rules.

6.4 Object detection and classification

Detection methods fall into two broad families. Two-stage detectors such as Faster R-CNN first propose candidate regions and then classify each one [27]. Single-stage detectors such as YOLO predict boxes and classes in one pass over the image, which makes them fast enough for real-time use [28]. Both rely on convolutional neural networks whose deep layers learn features directly from labelled examples, and deeper architectures such as residual networks made training such models much more reliable [29]. For driving, the purpose is to produce a list of road users and signs with positions. The main practical concern is the balance between speed and accuracy, since a detector that is accurate but slow reacts too late.

6.5 Semantic segmentation

Semantic segmentation labels each pixel rather than drawing boxes. A fully convolutional network replaces the final classification layers of a standard network with convolutional ones so that it outputs a class map at the image's resolution [17], and encoder–decoder designs such as U-Net add skip connections to recover fine boundaries [18]. In a driving scene it identifies the drivable surface, the sidewalk and the region occupied by vehicles, which is especially valuable where there are no lane lines. Its weakness is that it treats all pixels of a class alike, so two vehicles side by side form one blob unless instance information is added.

6.6 Motion estimation and object tracking

Optical flow estimates how each pixel moves between two frames. It rests on the brightness-constancy assumption that a point keeps its intensity as it moves, which gives the constraint $I_x u + I_y v + I_t = 0$ for a pixel with flow (u, v) [30]. Because this is one equation with two unknowns, methods add an extra assumption: Horn and Schunck assume smooth flow across the image [30], while Lucas and Kanade assume constant flow in a small window [31]. Tracking then follows detected objects over time. A Kalman filter predicts where an object will be and corrects the prediction with each new measurement [20], and a tracker such as SORT matches predictions with new detections using overlap between boxes [21]. For driving, these methods give relative speed and heading, which are needed to judge whether another car is braking or a pedestrian is about to cross.

6.7 Deep-learning-based visual perception

Deep learning ties the previous ideas together. A convolutional or transformer-based network learns its own filters from large numbers of labelled images, and one shared backbone can feed several output heads, for example detection, lane geometry and depth, an approach known as multi-task learning [14, 32]. For a multi-camera vehicle the networks must also combine views. Public research describes bird's-eye-view and occupancy representations that fuse several cameras and several moments in time [11, 33], and Tesla's occupancy network is one publicly presented example of this family [10, 11]. Deep models handle variation far better than hand-written rules, but they are harder to inspect, they depend on their training data, and they can fail in ways that are difficult to predict, which is why evaluation (Section 8) is so important.

6.8 A classical lane-finding pipeline in OpenCV

The techniques above combine into a pipeline that is a standard exercise in Python image-processing courses. It is a teaching example and is not claimed to be used by Tesla. The steps are as follows. First, convert the frame to grayscale and apply a Gaussian blur. Second, run *cv2.Canny* with thresholds tuned on sample frames. Third, mask a region of interest, typically a trapezoid in the lower half of the image where the road is expected, so that edges from trees, buildings and the sky are discarded. Fourth, apply the probabilistic Hough transform (*cv2.HoughLinesP*), which finds line

segments by letting edge pixels vote for the line parameters they are consistent with [5]. Fifth, split the segments into left and right groups by slope, average them, and extend the two lines to produce lane boundaries.

On clear, straight highway footage this works surprisingly well, which makes it a good way to learn how the operations interact. Its failure modes are equally instructive. Shadows of trees, tar seams, kerbs and the edges of other vehicles produce edges that look like lane lines. Faded paint produces weak gradients that the thresholds discard. Curves are poorly described by straight lines, a hill or a change in camera pitch moves the road out of the fixed region of interest, and the method has no idea what a lane line *is*, so a guardrail can be mistaken for one. These weaknesses are the practical reason why current lane-finding research uses learned models that classify and group pixels [16], and why a manufacturer that wants to work in a wide variety of conditions would be expected to rely on learning instead of fixed rules.

7. Practical Applications in Tesla Vehicles

7.1 Publicly documented applications

Tesla's documentation ties several features to camera input. Traffic-Aware Cruise Control maintains a set speed and following distance from a vehicle ahead if there is one [8]. Full Self-Driving (Supervised) is described as attempting to drive to a destination by following curves in the road, stopping at and negotiating intersections, making turns, using roundabouts and entering and leaving highways, and it uses inputs from cameras at the front, rear, left and right to build a model of the surroundings [9]. The same page states that the feature depends on the cameras' ability to detect traffic lights, stop signs and road markings, and that it mainly uses visual data, with map data improving accuracy [9]. In image-processing terms these statements correspond to recognising road users and signals, interpreting lane markings and road geometry, and detecting obstacles in the vehicle's path.

A second example is close-range obstacle sensing. When Tesla removed ultrasonic sensors, it stated that its vision-based occupancy network replaced the inputs they produced, and that some parking-related features were limited for a period on vehicles delivered without them [1]. This is a good example of a task that was previously solved with a dedicated range sensor being re-expressed as an image-interpretation problem. Tesla also states that Model 3 and Model Y with Tesla Vision have maintained or improved their active-safety ratings in the U.S. and Europe compared with radar-equipped vehicles [1]. This is the manufacturer's own claim, and I have not verified it independently.

The manual also shows how much the system depends on the condition of the cameras. It asks that cameras be clean and unobstructed, explains that the cameras must calibrate on roads with highly visible lane markings, and warns that dirt, rain and faded markings can affect performance [2, 13]. If a camera is blocked or blinded, the vehicle displays a message and the features may be unavailable [13]. These details connect the image-processing concepts of this report to everyday ownership: a smudge on a lens is a real degradation of the input image.

7.2 Driver assistance versus autonomous driving

The SAE J3016 standard defines six levels of driving automation, from Level 0 (no automation) to Level 5 (full automation in all conditions). In Levels 1 and 2 the human driver remains responsible for supervising the system and for the driving task as a whole, while from Level 3 upward the automated system performs the driving task within a defined domain [34]. NHTSA's documents record that Tesla itself described Autopilot as an SAE Level 2 driver-assistance system [3]. Tesla's own manual states that its self-driving features are advanced driver-assistance features that do not make the vehicle fully autonomous or replace the driver, and that the driver must pay attention and be ready to take immediate action [8]. The word "Supervised" in the current feature name reflects the same point [9].

This distinction matters for how results are judged. A fully autonomous system would have to cope with every situation in its operating domain without human help, while a supervised system is allowed to hand control back to a human who is expected to be paying attention. Judging a Level 2 system as if it were autonomous, or using it as if it were, is a misunderstanding of what it is. Features also differ by vehicle, hardware, software version and region, and the manual says a vehicle may not be equipped with a given feature or may not operate exactly as described [9]. Any statement about a specific capability should therefore be tied to a specific date, vehicle and software release, and this report's statements reflect the documentation I accessed in October 2026.

7.3 An illustrative walk-through: approaching a stop sign

To connect the stages of Section 5 to a real function, consider a car approaching a stop sign on a city street. Tesla's manual says only that Full Self-Driving (Supervised) depends on the cameras detecting stop signs, traffic lights and road markings [9]. The steps below are my own illustration of how the general stages could contribute, not a description of Tesla's software. The front cameras capture frames, and the image is normalised for brightness. A detector finds a red octagonal sign and classifies it as a stop sign, while a segmentation or lane model finds the stop line and the road ahead. Tracking over several frames confirms that the sign is stable, rejecting a one-frame false alarm, and the growth of the sign in the image together with the vehicle's own speed gives an estimate of distance. Scene interpretation then combines the sign, the stop line, any crossing traffic and the road layout, and passes the result to the planning and control functions

so that the vehicle slows. If the sign is hidden by a truck or covered by glare, the same chain breaks, which is why the manual reminds the driver to stay ready to act [2].

8. Data Collection, Training and Model Evaluation

Tesla has not published the size or composition of its training data, its model specifications or its internal accuracy figures, and this report does not guess them. What follows is the general process by which a camera-perception model is built and judged, together with public examples a student can work with in Python.

8.1 Datasets and annotation

A model learns from images whose contents have been labelled by people or by semi-automatic tools. Labels can be class names, bounding boxes, per-pixel masks, lane geometry or three-dimensional boxes. Public driving datasets such as KITTI, Cityscapes and nuScenes show what this looks like and how much work it takes [19, 35, 36]. Diversity is the main requirement. A dataset recorded in one city in good weather produces a model that is poor elsewhere, a problem known as dataset bias [37]. A useful dataset therefore needs different countries, road types, lighting, weather, camera positions and traffic behaviour. Annotation quality matters as much as quantity, since inconsistent labels teach inconsistent behaviour. Tesla says only, in general terms, that its occupancy network will keep improving over time [1], and it has not published how it collects or labels its data.

8.2 Training, validation and testing

The data are normally split into three parts. The *training set* adjusts the network's weights. The *validation set* is used during development to choose settings such as the learning rate or architecture and to notice overfitting, when the model memorises its training images. The *test set* is kept untouched until the end to estimate how well the model will do on new data. If the test set leaks into development, the reported results look better than the true performance. For driving, there is an extra requirement that the split should be made by location or by drive and not by individual frame, because consecutive frames of one drive are almost identical, and a random frame split would let the model see near-copies of its test images.

8.3 Evaluation measures

Different tasks need different measures (Table B2). For detection, a predicted box counts as a true positive if it overlaps a ground-truth box enough. Overlap is measured by **intersection over union**:

$$IoU = |A \cap B| / |A \cup B|,$$

where A is the predicted region and B is the true one [38]. With a chosen IoU threshold, precision and recall are

$$Precision = TP / (TP + FP), \quad Recall = TP / (TP + FN).$$

Precision answers "of the things the model reported, how many were real?" and recall answers "of the real things, how many did the model find?" In a safety setting the two errors have different costs: a false positive may cause unnecessary braking, while a false negative may mean a missed pedestrian. Average precision summarises the precision–recall trade-off over confidence levels and is averaged over classes (mAP) in benchmarks such as PASCAL VOC and COCO [38, 39]. Segmentation is scored by the mean IoU over classes [19], and tracking by measures such as multiple-object tracking accuracy, which counts misses, false alarms and identity switches [40]. Finally, **inference latency**, the time the system takes from receiving an image to producing its output, is as important as accuracy, because a correct answer that arrives late is of little use at road speed. Latency depends on the model and on the in-vehicle computer, and Tesla has not published such figures for its production systems.

A small hypothetical example shows how the two error types trade off. The numbers are invented for teaching and are not Tesla results. Suppose a pedestrian detector is tested on 100 real pedestrians. At a strict confidence threshold it finds 90 of them (TP = 90, FN = 10) and raises 10 false alarms (FP = 10), giving precision 90/100 = 0.90 and recall 90/100 = 0.90. If the threshold is lowered it finds 97 pedestrians but raises 40 false alarms, so recall rises to 0.97 while precision falls to 97/137 ≈ 0.71. Neither setting is simply "better". The right choice depends on the cost of each error and on what the rest of the system does with a detection, for example whether a second check can filter out false alarms. This is why reporting a single number for a perception model, without the threshold and the conditions, says very little about its safety.

8.4 Rare situations and difficult conditions

An average score hides what matters most. A model can reach a high mean accuracy on common scenes and still fail on the rare ones, because rare cases contribute few examples to both training and testing. Recognising this "long tail" leads to practices such as evaluating separately by condition (night, rain, glare, tunnels), deliberately collecting more examples of difficult cases, and testing with image corruptions such as fog, noise and blur [41]. Fairness is part of the same issue. One study of public object-detection models found that pedestrian detection performed differently for different skin-tone groups, which shows why dataset composition needs auditing [42]. For a safety-critical product, reporting one overall percentage is not enough. The numbers must be broken down by the conditions that cause harm.

8.5 Training practicalities

Several practical issues shape how a perception model is trained, and each can be explored on a small scale in Python. The loss function must match the task: cross-entropy for classification and per-pixel segmentation, a box-regression term for localisation, and overlap-based terms such as the Dice loss when classes are small [14]. Class imbalance is severe in road scenes, because road and sky fill most pixels while pedestrians and signs fill very few, so unweighted training can produce a model that does well on average and badly on the objects that matter. *Data augmentation* creates new training examples by changing brightness, adding noise or blur, and applying small shifts, which improves robustness to the conditions in Section 4. It has to be used carefully, however: a horizontal flip changes which side of the road traffic drives on and mirrors the text on signs. *Transfer learning* starts from a network pretrained on a large general dataset and fine-tunes it, which is how students typically get good results from limited data [29]. Label noise is a further problem, since human annotators disagree on where a boundary lies or whether a distant blob is a pedestrian. Finally, a model that is retrained must be re-tested on the whole benchmark and not only on the cases it was meant to fix, because an improvement in one condition can cause a regression in another.

8.6 Evaluation in practice

In practice evaluation goes beyond a single test run. Teams typically keep a fixed regression suite of difficult scenes, so that every new model version can be compared with the previous one on the same cases, and they break results down by scenario, such as night, rain, tunnels, construction zones and dense urban traffic. Offline testing on recorded video is cheap and repeatable but cannot show how the system affects the vehicle's behaviour, since a recording does not respond to the car's decisions. Closed-loop testing in simulation [43] and on test tracks adds that missing feedback, and real-world driving then reveals what neither of the other two predicted. Standards for the safety of the intended functionality encourage exactly this kind of scenario-based argument, in which the manufacturer shows that known hazardous situations have been identified and tested and that the remaining risk has been judged acceptable [44]. For students, the practical lesson is to evaluate on realistic splits, report per-condition results, and be explicit about what a test cannot show.

9. Benefits and Business Value

Visual awareness. Cameras around a vehicle can watch in all directions at once and never get tired or distracted, and a well-trained model can apply the same attention to the hundredth minute of a drive as to the first. Tesla describes the purpose of its self-driving features as making driving more convenient and less stressful [8]. **Convenience.** Features such as adaptive cruise control and lane keeping reduce the workload of long, monotonous drives, provided the driver remains attentive.

Software-based improvement. Because the perception is implemented largely in software, it can be improved after the vehicle is sold. Tesla says its occupancy network will continue to improve over time [1], and a camera-only hardware platform means that one improved model can be rolled out to many vehicles with the same sensors. This is a real advantage compared with a feature fixed in hardware. **Support for driver-assistance functions.** Tesla reports that its camera-based vehicles have maintained or improved active-safety ratings and perform better on pedestrian automatic emergency braking than their radar-equipped predecessors [1]. This is a company statement, and independent test data would be needed to confirm it for any given model year.

Business value. For the company, a single sensor type simplifies hardware and manufacturing and lets the engineering effort concentrate on one data source and one family of models. This is a reasonable inference, but Tesla's own support page frames the change in terms of safety and software rather than cost [1], so I treat cost savings as an interpretation and not as a documented fact. Since a vision-based driver-assistance package is also sold as an optional extra, perception quality is tied directly to customer trust and to regulatory standing.

Reading safety claims critically. Claims that a driver-assistance system improves safety need careful reading. Comparisons between vehicles with and without a feature can be distorted by differences in road type, because motorway driving is generally safer per kilometre than city driving, by differences in the drivers who choose to buy and use the feature, and by differences in how crashes are reported. A fair comparison has to control for these factors, and a figure published by a manufacturer without its method cannot be checked. Tesla's statement about active-safety ratings [1] is easier to interpret, because it appears to refer to ratings from standardised tests, but even such tests cover a limited set of scenarios and do not represent the rare and degraded conditions discussed in Section 8.4. A balanced reading therefore treats the benefits as plausible and partly supported, and treats the size of the real-world benefit as something that independent data must establish.

Balancing the benefits. These advantages have to be weighed against the limitations in Section 10. A camera-only approach depends on image quality, and NHTSA has opened an engineering analysis into whether FSD detects and warns the driver about degraded visibility from glare or airborne obscurants [4]. Convenience can also be a risk if it encourages drivers to relax. Table B3 summarises the main benefits, the matching limitations and possible mitigations.

10. Challenges, Limitations and Ethical Considerations

10.1 Visibility, weather and occlusion

Poor visibility is the best-documented limitation. Tesla's manual lists heavy rain, snow, fog and hail, obstructed or damaged cameras, and faded lane markings as factors that can limit performance [2, 13]. The regulator's concern is more specific. NHTSA opened a preliminary evaluation in October 2024 after crashes in reduced visibility, and in March 2026 it upgraded the investigation to an engineering analysis. The agency stated that it was evaluating whether the system fails to detect a degraded state, such as in glare and airborne obscurants, and to warn the driver [4]. An investigation is not a finding that a defect exists, and the outcome was not known when I wrote this report. It does show that detecting "my own vision is impaired" is a perception task in its own right: a system that cannot tell when its images are unreliable cannot hand control back in time.

The image-formation reasons behind these failures are worth stating. A sensor can record only a limited range of brightness between its darkest and brightest usable values. Looking towards a low sun, the bright region saturates, meaning that all detail is lost in it, and light scattering inside the lens spreads a veil of flare across neighbouring pixels, lowering contrast over a large part of the image. Techniques such as combining several exposures into a high-dynamic-range image can recover detail [6], but they take time and cannot restore information that was never recorded. Fog and airborne dust cause a similar loss of contrast, because they scatter light between the scene and the camera. Image processing can stretch the remaining contrast, as in Section 6.2, but that also amplifies noise. When the information is gone, better algorithms alone cannot bring it back, and the system must recognise the situation and respond conservatively.

10.2 False detections and missed objects

Every detector makes mistakes. A false detection, such as mistaking a shadow, bridge or reflection for an obstacle, can cause unnecessary braking. A missed detection, particularly of a small, partly hidden or unusual object, can mean no braking at all. The two cannot both be removed by adjusting a threshold, since lowering it catches more objects but raises false alarms (Section 8.3). Rare objects are especially risky because the model may have seen few or no examples of them.

10.3 Dataset bias, privacy and cybersecurity

If training data under-represent some people, regions or conditions, performance will be uneven, and public studies have shown measurable differences between groups in object-detection models [37, 42]. Privacy is a second concern, because cameras that observe the road also capture faces, number plates and homes. How such data are stored, anonymised, shared and used for training is governed by regulation and company policy, and a responsible design minimises what is collected and what is kept. Tesla's data practices are outside the scope of this report. Cybersecurity also matters. A perception system can be attacked through its inputs: researchers have shown that small, physically placed stickers on a stop sign can make a classifier misread it [45]. Connected vehicles are also exposed through their software update channels, so updates must be authenticated and validated before release.

10.4 Driver overreliance and safety accountability

A supervised system works only if the supervisor is paying attention. In December 2023, Tesla filed a recall report on its Autopilot controls with NHTSA, stating that the prominence and scope of the system's controls might not be enough to prevent driver misuse of the Level 2 feature [12]. NHTSA's later review reports that its investigation identified at least thirteen crashes involving one or more fatalities, and many more with serious injuries, in which foreseeable driver misuse played an apparent role [3], and the recall covered about 2.03 million vehicles [46]. These findings concern driver engagement, and they do not show that the cameras failed. They do show that the quality of image processing cannot be separated from how the driver interacts with the system. Tesla's manual puts the responsibility on the driver, who must stay in control at all times [2, 8], while regulators have examined whether the system's design does enough to keep that responsibility realistic. Accountability is therefore shared among the manufacturer, who designs the system and its safeguards, the driver, who must supervise it, and the regulator, who sets and enforces the rules.

10.5 Mitigation strategies

Several measures reduce these risks, and most of them are visible in the public documents. At the hardware level, the manual tells drivers to keep cameras clean and to complete calibration [13]. In software, the system can monitor its own image quality and warn when a camera is blocked [13], although the 2026 NHTSA investigation is examining whether such detection works well enough in glare and airborne obscurants [4]. Training can emphasise rare and degraded cases and evaluate by condition (Section 8.4). Redundancy is an option: other developers combine cameras with radar or lidar, and datasets such as nuScenes include these sensors [36], whereas Tesla has chosen to rely on cameras [1]. Whether that choice is sufficient for supervised driving is a question that regulators and researchers are still examining. Finally, human factors remedies such as stricter attention checks were part of the 2023 recall [12]. In every case, the driver's continuing attention and compliance with the vehicle's instructions remain essential, and the mitigations reduce risk without removing it.

10.6 Transparency and public expectations

Perception systems are difficult for outsiders to judge, because the models, data and internal test results belong to the manufacturer. NHTSA's investigation of reduced-visibility crashes, for example, relied on crash reports and on materials provided by Tesla [4], and that reliance on company data is typical of this field. Independent researchers can work with public datasets and benchmarks, but those do not reproduce a manufacturer's own camera hardware and fleet conditions. This asymmetry makes clear, honest communication more important. Feature names, owner's manuals and in-car messages all shape what drivers believe the system can do, and the addition of "(Supervised)" to the feature name and the repeated warnings in the manual are attempts to set that expectation correctly [8, 9]. The 2023 recall discussion shows that regulators judge not just the algorithm but whether the interface makes the driver's role clear enough [12]. For a student, the lesson is that evaluation has an ethical dimension: a result is only meaningful if it is reported in a way that lets others judge where the system should and should not be trusted.

11. Future Scope

This section lists plausible technical directions based on published research. They are not confirmed Tesla product plans, and I have not used statements by company executives as evidence of what will be delivered.

Better visual perception and temporal understanding. Research on bird's-eye-view networks and spatiotemporal transformers aims to build a consistent three-dimensional scene from several cameras over time, which should help with occlusion and with estimating motion [11, 33]. Longer temporal memory could allow a system to remember that a pedestrian was behind a parked van a moment ago. **Neural-network efficiency and edge computing.** Real-time driving requires inference on the vehicle itself, so model compression techniques such as pruning and quantisation are important for fitting larger models within a fixed latency and power budget [47].

Simulation and synthetic data. Open simulators such as CARLA let developers create rare scenarios, such as sudden pedestrians or heavy rain, at low cost and without risk [43]. Their main limitation is the gap between synthetic and real images, so simulation complements real-world data and does not replace it. **Model validation.** Standards such as ISO 21448, which addresses the safety of the intended functionality, deal with hazards that arise even when the system works as designed, such as perception limits in unusual conditions [44]. Systematic scenario-based testing and condition-specific reporting (Section 8.4) are likely to become more important as regulators ask for evidence. **Safety monitoring.** Improved self-assessment of image quality, better driver monitoring, and clearer communication of system limits all address problems described in Section 10 [4, 12]. Taken together, these directions would make perception more capable, but none of them removes the need to treat a supervised system as supervised.

Interpretability and regulation. Another direction is making deep perception models easier to examine, for example by visualising which image regions influenced a decision, so that engineers and regulators can understand why a failure occurred [14]. This links to standards and regulation: as driver-assistance systems spread, evidence about performance under defined conditions is likely to be requested more often, and the experience of the recall and investigations discussed in Section 10 suggests that attention to degraded conditions and driver engagement will continue [4, 12]. How far any of this develops for Tesla specifically is not something I can state from public evidence.

12. Conclusion

This case study shows that image processing and computer vision are the foundation of Tesla's current driver-assistance features. Tesla's own documents describe cameras around the vehicle, a transition away from radar and ultrasonic sensors, and an occupancy network to estimate nearby space [1, 9]. The general techniques in this report, from grayscale conversion and filtering to edge detection, segmentation, tracking and deep networks, explain how each part of a camera-based pipeline can work. The Python experiment in Figure A3 shows, in a small way, why classical edge rules need smoothing and careful thresholds, and why learned models are used where scenes vary so much.

The case also shows the limits of vision in a safety-critical setting. Image quality depends on lens cleanliness, calibration, light and weather, and the failure cases are often the rare ones, which are the hardest to collect, test and evaluate [2, 41]. Regulatory documents record concerns about both degraded-visibility detection and driver engagement [4, 12], and the features remain Level 2 systems that need a driver who is paying attention [8, 34]. A balanced view is that camera-based perception is powerful, scalable and improving, but it is not a substitute for human supervision, independent validation or transparency about limits. For a student of image processing the lesson is that accuracy on a benchmark is only one part of the answer, and that reliability under the worst conditions determines whether a vision system can be trusted on the road.

Several extensions would suit follow-up work in Python. A student could run the lane-finding pipeline of Section 6.8 on public dashcam or KITTI frames and measure where it breaks [35], train a small segmentation network on Cityscapes and report mean IoU by class [19], or add synthetic fog and noise to test images and track how precision and recall change [41]. Such experiments would produce original, measurable results about general techniques, which is the honest route to evidence in a field where the production systems cannot be inspected.

13. References

[1] Tesla, Inc., “Tesla Vision update: Replacing ultrasonic sensors with Tesla Vision,” Tesla Support. [Online]. Available: https://www.tesla.com/en_eu/support/transitioning-tesla-vision (accessed Oct. 27, 2026).

[2] Tesla, Inc., “Self-Driving limitations and warnings,” *Model Y Owner’s Manual*. [Online]. Available: https://www.tesla.cn/ownersmanual/modely/en_gb/GUID-E5FF5E84-6AAC-43E6-B7ED-EC1E9AEB17B7.html (accessed Oct. 27, 2026).

[3] NHTSA Office of Defects Investigation, “Recall query RQ24-009 opening resume: Recall 23V838 remedy effectiveness,” 2024. [Online]. Available: https://static.nhtsa.gov/odi/inv/2024/INOA-RQ24009-12046.pdf (accessed Oct. 27, 2026).

[4] NHTSA Office of Defects Investigation, “Investigation PE24031 close resume: FSD collisions in reduced roadway visibility conditions,” Mar. 18, 2026. [Online]. Available: https://static.nhtsa.gov/odi/inv/2024/INCLA-PE24031-87755.pdf (accessed Oct. 27, 2026).

[5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[6] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022. [Online]. Available: https://szeliski.org/Book/ (accessed Oct. 27, 2026).

[7] G. Bradski, “The OpenCV library,” *Dr. Dobb’s J. Softw. Tools*, 2000.

[8] Tesla, Inc., “About Self-Driving,” *Model S Owner’s Manual*. [Online]. Available: https://www.tesla.cn/ownersmanual/models/en_ca/GUID-101D1BF5-52D2-469A-A57D-E7230BBEE94B.html (accessed Oct. 27, 2026).

[9] Tesla, Inc., “Full Self-Driving (Supervised),” *Model Y Owner’s Manual*. [Online]. Available: https://www.tesla.com/ownersmanual/modely/en_au/GUID-2CB60804-9CEA-4F4B-8B04-09B991368DC5.html (accessed Oct. 27, 2026).

[10] A. Elluswamy, keynote presentation on occupancy networks, Workshop on Autonomous Driving (WAD), IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Jun. 2022. (Recorded conference talk, cited together with a published survey that summarises it.)

[11] Y. Shi *et al.*, “Grid-centric traffic scenario perception for autonomous driving: A comprehensive review,” arXiv:2303.01212, 2023. [Online]. Available: https://arxiv.org/pdf/2303.01212 (accessed Oct. 27, 2026).

[12] U.S. National Highway Traffic Safety Administration (NHTSA), “Part 573 safety recall report 23V-838: Autopilot controls insufficient to prevent misuse,” Tesla, Inc., Dec. 2023. [Online]. Available: https://static.nhtsa.gov/odi/rcl/2023/RCLRPT-23V838-8276.PDF (accessed Oct. 27, 2026).

[13] Tesla, Inc., “Cameras,” *Model Y Owner’s Manual*. [Online]. Available: https://www.tesla.com/ownersmanual/modely/en_il/GUID-682FF4A7-D083-4C95-925A-5EE3752F4865.html (accessed Oct. 27, 2026).

[14] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[15] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[16] D. Neven, B. De Brabandere, S. Georgoulis, M. Proesmans, and L. Van Gool, “Towards end-to-end lane detection: An instance segmentation approach,” in *Proc. IEEE Intell. Veh. Symp. (IV)*, Changshu, China, 2018, pp. 286–291.

[17] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Boston, MA, USA, 2015, pp. 3431–3440.

[18] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Med. Image Comput. Comput.-Assist. Intervent. (MICCAI)*, LNCS vol. 9351, 2015, pp. 234–241.

[19] M. Cordts *et al.*, “The Cityscapes dataset for semantic urban scene understanding,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 3213–3223.

[20] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.

[21] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Phoenix, AZ, USA, 2016, pp. 3464–3468.

[22] C. Tomasi and R. Manduchi, “Bilateral filtering for gray and color images,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Bombay, India, 1998, pp. 839–846.

[23] OpenCV, “Sobel derivatives,” OpenCV 4.11.0 tutorials. [Online]. Available: https://docs.opencv.org/4.11.0/d2/d2c/tutorial_sobel_derivatives.html (accessed Oct. 27, 2026).

[24] OpenCV, “Laplace operator,” OpenCV 4.1.1 tutorials. [Online]. Available: https://docs.opencv.org/4.1.1/d5/db5/tutorial_laplace_operator.html (accessed Oct. 27, 2026).

[25] J. Canny, “A computational approach to edge detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986, doi: 10.1109/TPAMI.1986.4767851.

[26] OpenCV, “Canny edge detection,” OpenCV tutorials. [Online]. Available: https://docs.opencv.org/5.x/da/d22/tutorial_py_canny.html (accessed Oct. 27, 2026).

[27] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[28] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[29] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.

[30] B. K. P. Horn and B. G. Schunck, “Determining optical flow,” *Artif. Intell.*, vol. 17, no. 1–3, pp. 185–203, Aug. 1981, doi: 10.1016/0004-3702(81)90024-2.

[31] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” in *Proc. 7th Int. Joint Conf. Artif. Intell. (IJCAI)*, Vancouver, BC, Canada, 1981, pp. 674–679.

[32] R. Caruana, “Multitask learning,” *Mach. Learn.*, vol. 28, no. 1, pp. 41–75, Jul. 1997.

[33] Z. Li *et al.*, “BEVFormer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, LNCS vol. 13669, 2022, pp. 1–18.

[34] SAE International, “Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles,” SAE Standard J3016_202104, Apr. 2021. [Online]. Available: https://www.sae.org/standards/content/j3016_202104/ (accessed Oct. 27, 2026).

[35] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Providence, RI, USA, 2012, pp. 3354–3361.

[36] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 2020, pp. 11621–11631.

[37] A. Torralba and A. A. Efros, “Unbiased look at dataset bias,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Colorado Springs, CO, USA, 2011, pp. 1521–1528.

[38] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) challenge,” *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010.

[39] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, LNCS vol. 8693, 2014, pp. 740–755.

[40] K. Bernardin and R. Stiefelhagen, “Evaluating multiple object tracking performance: The CLEAR MOT metrics,” *EURASIP J. Image Video Process.*, vol. 2008, Art. no. 246309, 2008.

[41] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1903.12261 (accessed Oct. 27, 2026).

[42] B. Wilson, J. Hoffman, and J. Morgenstern, “Predictive inequity in object detection,” arXiv:1902.11097, 2019. [Online]. Available: https://arxiv.org/abs/1902.11097 (accessed Oct. 27, 2026).

[43] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proc. 1st Conf. Robot Learn. (CoRL)*, PMLR vol. 78, 2017, pp. 1–16.

[44] International Organization for Standardization, “Road vehicles — Safety of the intended functionality,” ISO 21448:2022. [Online]. Available: https://www.iso.org/standard/77490.html (accessed Oct. 27, 2026).

[45] K. Eykholt *et al.*, “Robust physical-world attacks on deep learning visual classification,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 1625–1634.

[46] NHTSA, “Recall quarterly report 23V838,” Tesla, Inc., updated Jul. 30, 2025. [Online]. Available: https://static.nhtsa.gov/odi/rcl/2023/RCLQRT-23V838-9544.pdf (accessed Oct. 27, 2026).

[47] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016. [Online]. Available: https://arxiv.org/abs/1510.00149 (accessed Oct. 27, 2026).

Appendix A — Figures

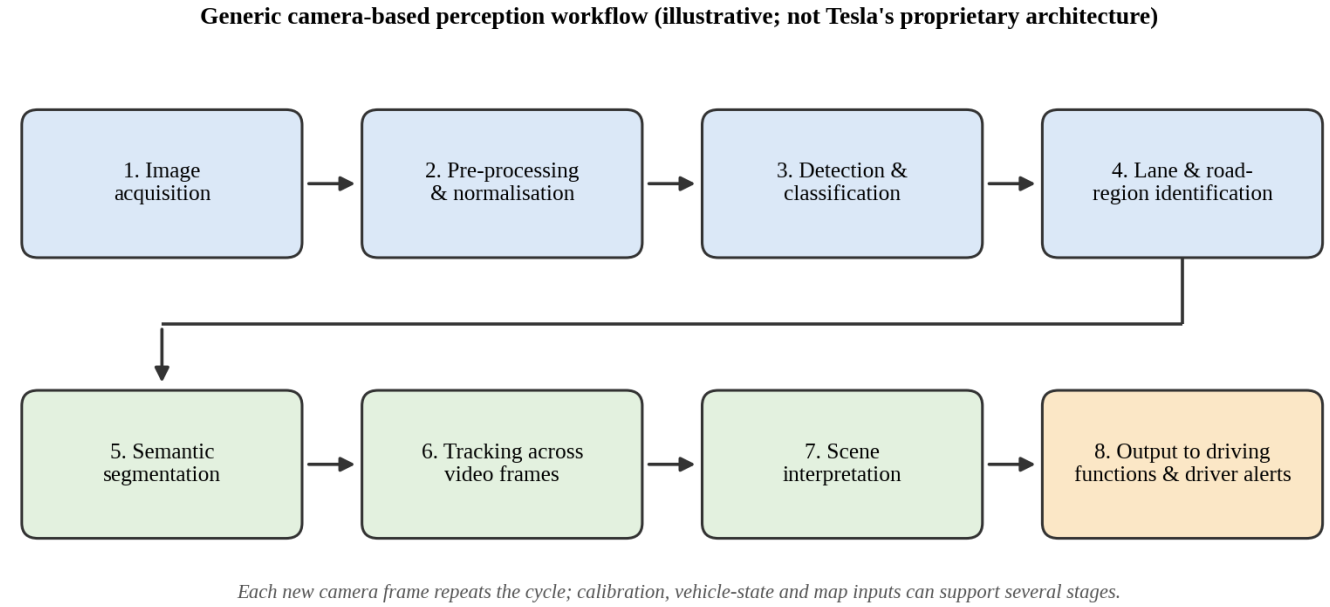


Figure A1. General image-processing workflow for a camera-based driving system. The diagram is an illustrative teaching model based on common computer-vision practice and is not Tesla's proprietary architecture.

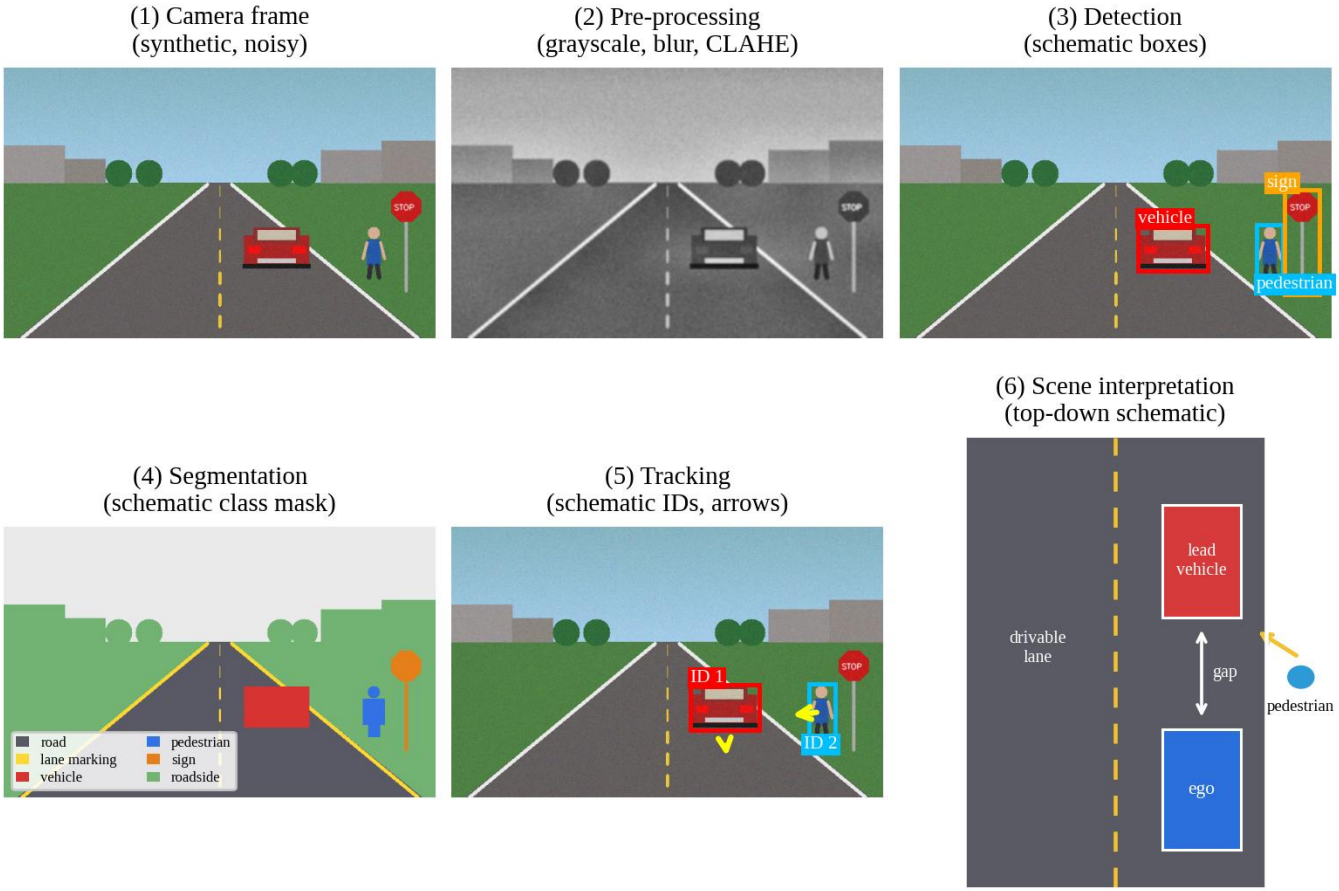


Figure A2. Illustrative stages of visual perception, from camera input to scene interpretation, shown on a synthetic road scene created for this report. Panels 1 and 2 are real image-processing operations (noise added; grayscale, Gaussian blur and CLAHE applied). Panels 3 to 6 are schematic annotations drawn from the known object positions of the synthetic scene. They are not outputs of a trained network or of any Tesla system.

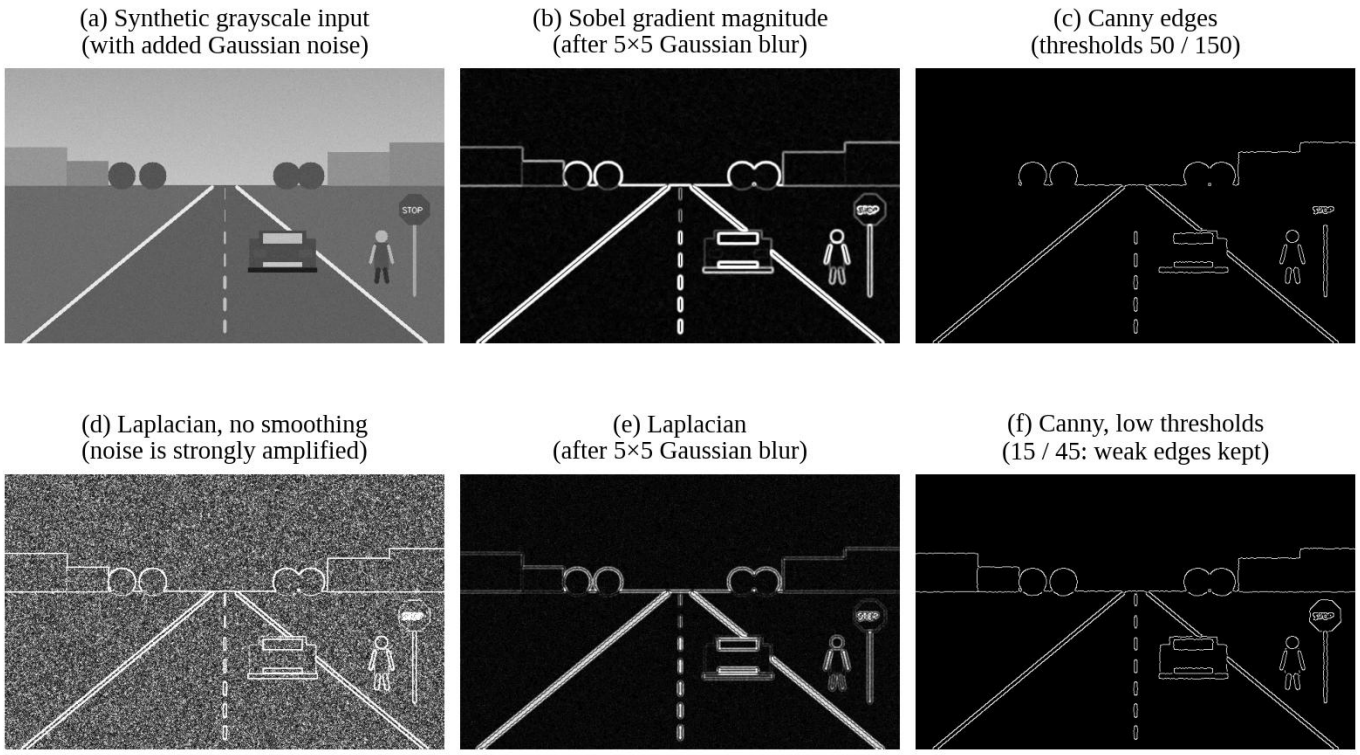


Figure A3. Comparison of edge-detection techniques on the synthetic noisy grayscale road image, computed with OpenCV in Python (cv2.Sobel, cv2.Laplacian, cv2.Canny; blur: 5×5 Gaussian, $\sigma = 1.4$). Panel (d) shows the Laplacian without smoothing, where noise dominates. The image is synthetic and the experiment says nothing about Tesla's production performance.

Appendix B — Tables

Table B1. Image-processing techniques and their purposes.

Technique	Purpose	How it could help interpret a driving scene	Typical Python / OpenCV entry point
Grayscale and colour-space conversion	Reduce data; separate brightness from colour.	Simplifies edge finding; hue-based search for white or yellow markings and for signal colours.	cv2.cvtColor (BGR→GRAY, HSV, YCrCb)
Filtering and noise reduction	Suppress sensor noise; improve contrast.	Cleans night-time or rainy frames before later stages; CLAHE brightens dim regions.	cv2.GaussianBlur, medianBlur, bilateralFilter, createCLAHE
Sobel gradient	Estimate horizontal and vertical intensity gradients.	Highlights lane-line and object boundaries and their orientation.	cv2.Sobel
Laplacian	Second-derivative edge response.	Finds fine edges, but is noise-sensitive and usually needs smoothing first.	cv2.Laplacian
Canny edge detection	Thin, connected edges using hysteresis thresholds.	Candidate lane lines and object outlines; input to line fitting.	cv2.Canny
Object detection and classification	Locate objects and assign classes.	Vehicles, pedestrians, cyclists and signs with positions and confidence.	Trained CNN detectors (for example via cv2.dnn)
Semantic segmentation	Label every pixel with a class.	Drivable surface, sidewalk, vehicles; useful where lane lines are absent.	Fully convolutional / encoder–decoder networks
Motion estimation and tracking	Estimate pixel or object motion over time.	Relative speed and heading; stable identities across frames.	cv2.calcOpticalFlowPyrLK, KalmanFilter
Deep-learning visual perception	Learn features and fuse several cameras and times.	Multi-camera scene representations such as occupancy or bird's-eye views.	PyTorch / TensorFlow models

Table B2. Evaluation metrics and their meanings.

Metric	Definition	What it tells us	Safety-related caution
Precision	TP / (TP + FP)	Share of reported objects that are real.	Low precision means false alarms, for example unnecessary braking.
Recall	TP / (TP + FN)	Share of real objects that were found.	Low recall means missed objects, which can be more dangerous.
Intersection over union (IoU)	$\|A \cap B\| / \|A \cup B\|$ for predicted region A and true region B	How well a predicted box or mask overlaps the truth.	Results depend on the chosen IoU threshold.
Average precision / mAP	Area under the precision–recall curve, averaged over classes	Summary of detector quality over confidence levels.	An average can hide poor results on rare classes or conditions.
Mean IoU (mIoU)	IoU averaged over all classes	Quality of semantic segmentation.	Small but critical classes, such as pedestrians, may be hidden by large classes.
Tracking accuracy (MOTA)	Accounts for misses, false positives and identity switches	Consistency of tracking over time.	Identity switches can break motion prediction.
Inference latency	Time from input image to output, in milliseconds	Whether the system is fast enough for real-time use.	Must be measured on the target in-vehicle hardware, and include worst cases.

Table B3. Benefits, limitations and possible mitigation strategies.

Area	Benefit	Limitation	Possible mitigation
Visual awareness	Cameras around the vehicle give continuous, all-round coverage.	Quality depends on clean lenses, lighting and weather [2].	Camera-cleaning guidance; self-monitoring of image quality with driver alerts.
Driving convenience	Less workload on long or repetitive drives.	Can encourage overreliance by the driver [12].	Driver-monitoring and attention checks; clear communication of limits.
Software improvement	Perception can be improved after sale and applied across a fleet.	A change can introduce new errors; results differ by version.	Staged roll-out, regression testing and condition-specific validation.
Single sensor type	Simpler hardware and a single data source.	No independent sensor to cross-check difficult visual scenes [4].	Better degradation detection; consider redundancy; restrict use in poor visibility.
Learned models	Cope with varied scenes better than hand-written rules.	Hard to inspect; weak on rare or unseen cases; biased data [42].	Diverse datasets; evaluation by condition; robustness testing.
Connectivity	Updates and fleet feedback.	Privacy exposure; attack surface for software and physical-world attacks [45].	Data minimisation; secure and authenticated updates; adversarial testing.